UD4 — Optimización, documentación y refactorización

Entornos de Desarrollo · 1.º DAW · 14 horas

Índice

1 Antes de empezar **2**
- 1.1 Qué vas a saber hacer al terminar esta unidad . . . 2
- 1.2 Por qué esta unidad va después de las pruebas . . . 2

2 Calidad del código y deuda técnica **3**
- 2.1 Qué es «código de calidad» . . . 3
- 2.2 Deuda técnica . . . 3

3 Code smells: los olores del código **4**
- 3.1 Los que más vas a encontrar . . . 4

4 Refactorización **6**
- 4.1 Definición . . . 6
- 4.2 Cuándo se refactoriza . . . 6
- 4.3 Catálogo de patrones de refactorización . . . 6
- 4.4 Ejemplo completo: refactorización paso a paso . . . 7
- 4.5 Refactorización asistida por el IDE . . . 8

5 Análisis estático y formateo **9**
- 5.1 Análisis estático . . . 9
- 5.2 Formateo: Prettier . . . 10

6 Documentación del código **12**
- 6.1 Los tres niveles de documentación . . . 12
- 6.2 Comentarios: cuándo sí y cuándo no . . . 12
- 6.3 JSDoc . . . 12
- 6.4 El README . . . 14

7 Control de versiones **16**
- 7.1 El problema que resuelve . . . 16
- 7.2 Conceptos . . . 16
- 7.3 Centralizado vs distribuido . . . 16
- 7.4 Git: los comandos que necesitas . . . 16
- 7.5 Mensajes de commit . . . 18
- 7.6 Flujos de trabajo . . . 18
- 7.7 Git dentro de VS Code . . . 19

8 Integración continua **20**
- 8.1 Qué es . . . 20
- 8.2 GitHub Actions . . . 20

9 Resumen de la unidad **21**

10 Autoevaluación **22**

1 Antes de empezar

1.1 Qué vas a saber hacer al terminar esta unidad

Cubre el **RA4: optimiza y documenta aplicaciones, aplicando buenas prácticas de mantenimiento**. Al acabarla tienes que ser capaz de:

- Explicar qué es la refactorización y por qué es peligrosa sin pruebas.
- Reconocer los *code smells* más habituales y aplicar el patrón de refactorización adecuado.
- Usar las herramientas de refactorización automática del IDE.
- Configurar un analizador estático (ESLint) y un formateador (Prettier).
- Documentar código con JSDoc y generar documentación navegable.
- Gestionar un repositorio Git: ramas, fusiones, conflictos, remoto, *pull requests*.
- Mantener la integridad de un repositorio y un historial legible.

1.2 Por qué esta unidad va después de las pruebas

En la numeración de la programación, UD4 aparece antes que UD3. Aquí se imparte al revés, y es deliberado: **refactorizar sin pruebas es cambiar código a ciegas**. La regla de oro de esta unidad —«no se refactoriza lo que no está cubierto por pruebas»— solo se puede aplicar de verdad si el alumnado ya tiene la batería de UD3 funcionando.

De modo que el orden real es: en UD3 construyes la red de seguridad; en UD4 la usas para atreverte a tocar el código, lo documentas y lo pones bajo control de versiones. Las dos unidades pertenecen a la misma evaluación, así que el cambio no afecta a la calificación.

2 Calidad del código y deuda técnica

2.1 Qué es «código de calidad»

Un código funciona o no funciona: eso lo decide la máquina. Que sea de calidad lo decide **la siguiente persona que tenga que tocarlo**, que dentro de seis meses probablemente serás tú.

Características de un código de calidad:

- **Legible**: se entiende leyéndolo, sin descifrarlo.
- **Mantenible**: cambiar algo no obliga a tocar quince sitios.
- **Probable** (*testable*): se puede probar por partes.
- **Reutilizable**: las piezas sirven en más de un sitio.
- **Eficiente**: no desperdicia recursos sin motivo.
- **Portable**: no depende de detalles del entorno concreto.

Idea clave

El código se escribe una vez y se lee decenas. Optimiza para la lectura, no para escribir menos teclas. Ese `if (!!x?.a?.b ?? c)` que te ha quedado tan ingenioso será un problema en marzo.

2.2 Deuda técnica

Metáfora de Ward Cunningham: tomar un atajo hoy es **pedir prestado tiempo**. Funciona, entregas antes, pero **pagas intereses**: cada cambio futuro sobre ese código cuesta más.

- **Deuda deliberada y prudente**: «sabemos que esto no es la mejor solución, pero hay que entregar el viernes; lo arreglamos en el sprint siguiente». Aceptable, si se apunta.
- **Deuda deliberada e imprudente**: «no tenemos tiempo de diseñar, a picar».
- **Deuda accidental y prudente**: «ahora que está hecho, vemos cómo deberíamos haberlo hecho». Inevitable y sana.
- **Deuda accidental e imprudente**: «¿qué es diseñar?».

Como toda deuda, si no se paga nunca, el proyecto acaba en bancarrota: llega un momento en que cualquier cambio pequeño rompe tres cosas y nadie se atreve a tocar nada.

3 Code smells: los olores del código

Un *code smell* no es un error: el programa funciona. Es un **síntoma** de que algo va a dar problemas. El catálogo clásico es de Martin Fowler (*Refactoring*, 1999).

3.1 Los que más vas a encontrar

3.1.1 1. Código duplicado

El peor de todos. Si el mismo bloque aparece en tres sitios, el día que haya que cambiarlo lo cambiarás en dos.

```
// MAL
function precioConIvaLibro(p)   { return p * 1.04; }
function precioConIvaComida(p)  { return p * 1.10; }
function precioConIvaGeneral(p) { return p * 1.21; }

// MEJOR
const IVA = { libro: 0.04, comida: 0.10, general: 0.21 };
function precioConIva(precio, tipo = 'general') {
  return precio * (1 + IVA[tipo]);
}
```

3.1.2 2. Método largo

Si una función no cabe en la pantalla, hace demasiadas cosas. Regla práctica: una función debe hacer **una** cosa y su nombre debe decir cuál.

3.1.3 3. Lista de parámetros larga

```
// MAL
crearTarea(titulo, descripcion, prioridad, fecha, etiqueta, usuario, hecha, color);

// MEJOR: objeto de parámetros
crearTarea({ titulo, descripcion, prioridad, fecha, etiqueta, usuario });
```

3.1.4 4. Números y cadenas mágicas

```
// MAL
if (usuario.rol === 3) { ... }
if (tarea.estado === 2) { ... }

// MEJOR
const ROL_ADMIN = 3;
const ESTADO = Object.freeze({ PENDIENTE: 1, EN_CURSO: 2, HECHA: 3 });
if (usuario.rol === ROL_ADMIN) { ... }
if (tarea.estado === ESTADO.EN_CURSO) { ... }
```

3.1.5 5. Nombres poco expresivos

```
// MAL
const d = new Date();
```

```
function proc(a, b) { ... }
let temp, aux, datos2;

// MEJOR
const fechaCreacion = new Date();
function calcularDiasRestantes(fechaLimite, fechaActual) { ... }
```

3.1.6 6. Condicionales anidados (*arrow code*)

```
// MAL
function procesar(tarea) {
  if (tarea) {
    if (!tarea.hecha) {
      if (tarea.fechaLimite) {
        if (tarea.fechaLimite < Date.now()) {
          return 'vencida';
        }
      }
    }
  }
  return 'ok';
}

// MEJOR: cláusulas de guarda
function procesar(tarea) {
  if (!tarea) return 'ok';
  if (tarea.hecha) return 'ok';
  if (!tarea.fechaLimite) return 'ok';
  return tarea.fechaLimite < Date.now() ? 'vencida' : 'ok';
}
```

3.1.7 7. Clase enorme (*God class*)

Una clase con 40 métodos y 25 atributos que lo hace todo. Se divide por responsabilidades.

3.1.8 8. Comentarios que explican código malo

```
// MAL
// comprueba si el usuario es mayor de edad y está activo y no está baneado
if (u.e >= 18 && u.a === 1 && u.b === 0) { ... }

// MEJOR: el código se explica solo y el comentario sobra
const puedeAcceder = usuario.edad >= 18 && usuario.activo && !usuario.baneado;
if (puedeAcceder) { ... }
```

Ojo

Un comentario que explica *qué* hace el código suele ser un síntoma. Los comentarios útiles explican **por qué**: «usamos una espera de 300 ms porque la API del banco rechaza más de 3 peticiones por segundo». Eso no se deduce del código.

4 Refactorización

4.1 Definición

> **Refactorizar** es modificar la estructura interna del código **sin alterar su comportamiento externo**, con el objetivo de hacerlo más legible, más simple o más fácil de mantener.

Las tres partes de la definición importan:

- **Estructura interna**: cambias cómo está escrito.
- **Sin alterar el comportamiento externo**: si antes devolvía 7, después devuelve 7. Si antes fallaba con `null`, después falla igual.
- **No es lo mismo que**: corregir errores (eso cambia el comportamiento), optimizar el rendimiento (aunque a veces coinciden) ni añadir funcionalidad.

Idea clave

Regla de oro: nunca refactorices sin pruebas automáticas. Si no tienes una red que te diga «sigue haciendo lo mismo», no estás refactorizando: estás reescribiendo a ciegas y cruzando los dedos. Por eso UD3 y UD4 son inseparables, y por eso UD3 va primero.

4.2 Cuándo se refactoriza

- **Regla de tres** (Fowler): la primera vez, lo haces. La segunda vez que haces algo parecido, te aguantas. La tercera vez, refactorizas.
- **Antes de añadir una función**: si el código actual dificulta el cambio, primero lo preparas.
- **Al arreglar un error**: si te ha costado encontrarlo, es que el código no era claro.
- **En la revisión de código**: cuando alguien tiene que preguntarte qué hace algo.

Y **cuándo no**: cuando el código va a desaparecer pronto, cuando hay que entregar en una hora, y cuando no tienes pruebas (véase la regla de oro).

4.3 Catálogo de patrones de refactorización

Patrón	Cuándo	Qué haces
Extraer función/método	Un trozo de código tiene entidad propia	Lo sacas a una función con nombre descriptivo
Incorporar función	Una función es tan trivial que estorba	La disuelves en quien la llama
Extraer variable	Una expresión es difícil de leer	Le das nombre a un trozo
Renombrar	El nombre no dice lo que hace	Lo cambias en todas partes
Sustituir número mágico por constante	Aparece un literal sin explicación	Constante con nombre
Descomponer condicional	Un `if` con condición compleja	Extraes la condición y los cuerpos a funciones
Consolidar expresión condicional	Varios `if` con el mismo resultado	Los unes en uno
Reemplazar condicional por polimorfismo	Un `switch` sobre el tipo de objeto	Cada tipo implementa su versión
Introducir cláusula de guarda	Anidamiento excesivo	Sales pronto de la función
Eliminar parámetro / Añadir parámetro	La firma no encaja	Ajustas la interfaz

Patrón	Cuándo	Qué haces
Introducir objeto de parámetros	Demasiados parámetros que van juntos	Los agrupas en un objeto
Mover método / campo	Está en la clase equivocada	Lo llevas a quien lo usa
Extraer clase	Una clase hace dos cosas	La partes en dos
Encapsular campo	Atributo público	Lo haces privado con acceso controlado

4.4 Ejemplo completo: refactorización paso a paso

Partimos de esto (todo mal a la vez, a propósito):

```
function f(l) {
  let r = '';
  for (let i = 0; i < l.length; i++) {
    if (l[i].h === false) {
      if (l[i].fl) {
        if (new Date(l[i].fl).getTime() < new Date().getTime()) {
          r += '[!] ' + l[i].t + ' (vencida el ' + l[i].fl + ')\n';
        } else {
          r += '[ ] ' + l[i].t + ' (para el ' + l[i].fl + ')\n';
        }
      } else {
        r += '[ ] ' + l[i].t + '\n';
      }
    } else {
      r += '[x] ' + l[i].t + '\n';
    }
  }
  return r;
}
```

Paso 1 — Renombrar (F2 en VS Code). `f` → `formatearListaTareas`, `l` → `tareas`, `r` → `salida`, `t` → `titulo`, `h` → `hecha`, `fl` → `fechaLimite`.

Paso 2 — Sustituir bucle por operación de colección.

```
function formatearListaTareas(tareas) {
  return tareas.map(formatearTarea).join('\n');
}
```

Paso 3 — Extraer función para la línea individual, con **cláusulas de guarda**.

```
function formatearTarea(tarea) {
  if (tarea.hecha) return `${MARCA.HECHA} ${tarea.titulo}`;
  if (!tarea.fechaLimite) return `${MARCA.PENDIENTE} ${tarea.titulo}`;
  if (estaVencida(tarea)) {
    return `${MARCA.VENCIDA} ${tarea.titulo} (vencida el ${tarea.fechaLimite})`;
  }
  return `${MARCA.PENDIENTE} ${tarea.titulo} (para el ${tarea.fechaLimite})`;
}
```

Paso 4 — Extraer función para la condición y **sustituir cadenas mágicas por constantes**.

```js
const MARCA = Object.freeze({ HECHA: '[x]', PENDIENTE: '[ ]', VENCIDA: '[!]' });

function estaVencida(tarea, ahora = new Date()) {
  return new Date(tarea.fechaLimite) < ahora;
}
```

Resultado: cuatro piezas pequeñas, con nombre, probables por separado. Y **exactamente el mismo comportamiento**.

Ojo

Fíjate en el parámetro `ahora = new Date()` de `estaVencida`. No es cosmético: sin él, esa función es **imposible de probar** porque su resultado depende de la hora a la que ejecutes la prueba. Refactorizar bien es, casi siempre, hacer el código más probable.

4.5 Refactorización asistida por el IDE

VS Code hace estas operaciones **de forma segura**, entendiendo el código, no buscando y reemplazando texto:

Acción	Cómo
Renombrar símbolo en todo el proyecto	F2
Extraer a función / método	Selecciona → `Ctrl+.` → *Extract to function*
Extraer a constante	Selecciona → `Ctrl+.` → *Extract to constant*
Convertir a función flecha / función normal	`Ctrl+.` sobre la función
Mover a nuevo fichero	`Ctrl+.` sobre la clase/función
Organizar imports	`Shift+Alt+O`
Ver todas las referencias	`Shift+F12`
Ir a la definición	F12

Idea clave

La diferencia entre F2 y `Ctrl+H`: F2 entiende el ámbito. Si tienes una variable `id` local en tres funciones distintas, F2 renombra solo la del ámbito en el que estás; `Ctrl+H` renombraría las tres y además la palabra «id» dentro de un comentario. **Nunca refactorices con buscar y reemplazar.**

Práctica

P4.1 — Rescate de código (2 h)
Se te entrega el fichero `sucio.js` (disponible en Moodle) con al menos **ocho** *code smells* distintos. Tu trabajo:

1. Ejecútalo y anota exactamente qué hace. Esa es tu referencia de comportamiento.
2. Crea una tabla: *olor detectado* → *línea* → *patrón de refactorización que aplicas*.
3. Refactoriza en **pasos pequeños**, ejecutando después de cada paso.
4. Usa las herramientas del IDE (F2, *Extract to function*) siempre que puedas.
5. Comprueba al final que el programa produce exactamente la misma salida.
6. Entrega: código original, código refactorizado, tabla y una captura del *diff*.

Se evalúa el proceso, no solo el resultado: quiero ver los pasos intermedios en el historial de Git.

5 Análisis estático y formateo

5.1 Análisis estático

Herramientas que examinan el código **sin ejecutarlo** y avisan de problemas: variables sin usar, comparaciones sospechosas, complejidad excesiva, funciones inalcanzables, posibles errores.

5.1.1 ESLint

El estándar en JavaScript. Configuración moderna en `eslint.config.js`:

```
import js from '@eslint/js';

export default [
  js.configs.recommended,
  {
    languageOptions: {
      ecmaVersion: 2024,
      sourceType: 'module',
      globals: { process: 'readonly', console: 'readonly' },
    },
    rules: {
      'no-unused-vars': 'warn',
      'no-console': 'off',
      'eqeqeq': ['error', 'always'],
      'prefer-const': 'error',
      'no-var': 'error',
      'complexity': ['warn', 8],
      'max-lines-per-function': ['warn', 40],
      'max-params': ['warn', 4],
      'camelcase': 'warn',
    },
  },
];
```

```
npm install -D eslint
npx eslint src           # analiza
npx eslint src --fix     # arregla lo que puede automáticamente
```

Las reglas tienen tres niveles: `"off"`, `"warn"` (avisa pero no falla) y `"error"` (devuelve código de error, rompe la construcción).

5.1.2 Métricas de código

Métrica	Qué mide	Valor razonable
Complejidad ciclomática	Número de caminos independientes por una función	< 10
Líneas por función	Tamaño	< 40
Número de parámetros	Acoplamiento de la firma	≤ 4
Profundidad de anidamiento	Legibilidad	≤ 3
Acoplamiento	Cuánto depende un módulo de otros	El mínimo
Cohesión	Cuánto tienen que ver entre sí las partes de un módulo	El máximo

La **complejidad ciclomática** se calcula fácil: 1 + número de puntos de decisión (`if`, `else if`, `for`, `while`, `case`, `&&`, `||`, `?:`, `catch`).

```
function clasificar(t) {          // 1
  if (t.hecha) return 'hecha';    // +1 = 2
  if (t.fecha && t.fecha < hoy)   // +1 (if) +1 (&&) = 4
    return 'vencida';
  for (const e of t.etiquetas) {  // +1 = 5
    if (e === 'urgente')          // +1 = 6
      return 'urgente';
  }
  return 'normal';
}
// Complejidad ciclomática = 6 → también es el número mínimo de casos de prueba
```

Idea clave

La complejidad ciclomática te da **el número mínimo de casos de prueba** que necesitas para cubrir todos los caminos de una función. Es el puente directo entre esta unidad y la UD3.

5.1.3 Otras herramientas

- **SonarQube / SonarLint**: análisis más profundo, detecta *code smells*, bugs y vulnerabilidades. SonarLint se integra en VS Code.
- **npm audit**: vulnerabilidades conocidas en dependencias.
- **TypeScript**: el análisis estático definitivo para JavaScript (tipos en tiempo de compilación). Lo veréis en 2.º.

5.2 Formateo: Prettier

ESLint dice si tu código está **bien**; Prettier decide cómo se **ve**. No discuten: se configuran juntos.

`.prettierrc`:

```
{
  "semi": true,
  "singleQuote": true,
  "printWidth": 90,
  "tabWidth": 2,
  "trailingComma": "all",
  "arrowParens": "avoid",
  "endOfLine": "lf"
}
```

Con `"editor.formatOnSave": true` en VS Code, se formatea solo al guardar. Ventaja real: se acabaron las discusiones de estilo en las revisiones de código y los *diffs* llenos de cambios de espacios.

Práctica

P4.2 — Calidad automática (2 h)
Sobre `gestor-tareas`:

1. Instala y configura ESLint con las reglas del ejemplo.
2. `npx eslint src` y cuenta cuántos avisos salen. Anótalo.
3. Corrige **todos**, a mano o con `--fix`. Justifica en un documento los que hayas corregido a mano.
4. Instala Prettier, crea `.prettierrc` y activa el formateo al guardar.

5. Añade los scripts `lint`, `lint:fix` y `format` al `package.json`.
6. Calcula **a mano** la complejidad ciclomática de las tres funciones más largas de tu proyecto. Compara con lo que dice la regla `complexity` de ESLint.
7. Ejecuta `npm audit` y explica qué te dice.

6 Documentación del código

6.1 Los tres niveles de documentación

Nivel	Para quién	Qué contiene	Dónde vive
De usuario	Quien usa el programa	Manual, ayuda, tutoriales, FAQ	Web, PDF, ayuda en línea
Técnica / de código	Quien mantiene el código	Qué hace cada función, parámetros, ejemplos, diagramas	En el propio código + generada
De proyecto	Equipo y cliente	Requisitos, diseño, plan de pruebas, actas	Gestor documental

6.2 Comentarios: cuándo sí y cuándo no

Comentarios que sobran:

```
i++;                        // incrementa i
// función que suma dos números
function sumar(a, b) { return a + b; }
```

Comentarios que valen oro:

```
// La API de Correos devuelve el código postal como número, perdiendo el 0
// inicial de las provincias 01-09. Lo restauramos aquí.
const cp = String(respuesta.cp).padStart(5, '0');

// TODO: sustituir por UUID cuando migremos a base de datos (issue #42)
const id = Date.now();

// HACK: Safari < 16 no soporta lookbehind en regex. Quitar en 2027.
```

Convenciones de marcado que las extensiones reconocen: `TODO`, `FIXME`, `HACK`, `NOTE`, `XXX`, `@deprecated`.

6.3 JSDoc

Es el estándar para documentar JavaScript. Escribes comentarios con una sintaxis concreta y obtienes **dos cosas**: mejor autocompletado en el IDE y documentación navegable generada.

```
/**
 * Representa una tarea del gestor.
 * @class
 */
export class Tarea {
  /**
   * Crea una tarea nueva.
   * @param {Object}  datos                - Datos de la tarea.
   * @param {number}  datos.id             - Identificador único.
   * @param {string}  datos.titulo         - Texto descriptivo.
   * @param {string} [datos.fechaLimite]   - Fecha límite en formato ISO (opcional).
   * @param {string[]} [datos.etiquetas=[]]  - Etiquetas asociadas.
   * @throws {TypeError} Si el título está vacío.
   */
  constructor({ id, titulo, fechaLimite, etiquetas = [] }) { /* ... */ }
```

```
  /**
   * Indica si la tarea ha superado su fecha límite.
   *
   * @param {Date} [ahora=new Date()] - Momento con el que comparar.
   * @returns {boolean} `true` si está vencida y no está hecha.
   *
   * @example
   * const t = new Tarea({ id: 1, titulo: 'Entregar', fechaLimite: '2026-01-01' });
   * t.estaVencida(new Date('2026-02-01')); // true
   */
  estaVencida(ahora = new Date()) { /* ... */ }
}
```

6.3.1 Etiquetas JSDoc más usadas

Etiqueta	Para qué
`@param {tipo} nombre - desc`	Un parámetro
`@returns {tipo} desc`	Lo que devuelve
`@throws {tipo} desc`	Excepciones que puede lanzar
`@example`	Ejemplo de uso
`@author`, `@version`, `@since`	Metadatos
`@deprecated`	Marcado como obsoleto
`@see`	Referencia cruzada
`@typedef` / `@property`	Definir un tipo compuesto
`@module`, `@class`	Estructura
`@private`, `@readonly`	Visibilidad

Corchetes = parámetro opcional: `@param {string} [nombre]`. Con valor por defecto: `@param {string} [nombre='sin título']`.

6.3.2 Generar la documentación

```
npm install -D jsdoc
npx jsdoc src -r -d docs
# abre docs/index.html
```

O con una plantilla más moderna:

```
npm install -D jsdoc clean-jsdoc-theme
npx jsdoc -c jsdoc.json
```

`jsdoc.json`:

```
{
  "source": { "include": ["src"], "includePattern": ".js$" },
  "opts": {
    "destination": "docs/",
    "recurse": true,
    "template": "node_modules/clean-jsdoc-theme",
    "readme": "README.md"
  },
```

```
  "templates": { "cleverLinks": true }
}
```

6.3.3 Equivalentes en otros lenguajes

Lenguaje	Herramienta	Comando
Java	**Javadoc**	`javadoc -d docs src/*.java`
JavaScript/TypeScript	JSDoc, TypeDoc	`npx jsdoc src -d docs`
Python	Sphinx, pydoc	`sphinx-build`
PHP	phpDocumentor	`phpdoc -d src -t docs`
C#	XML docs + DocFX	`docfx`

Ojo

Para el examen: **Javadoc** es la herramienta de Java y usa las mismas etiquetas (`@param`, `@return` —en singular—, `@throws`). Los IDE (Eclipse, IntelliJ, VS Code) generan el esqueleto del comentario automáticamente al escribir `/**` y pulsar Intro encima de un método.

6.4 El README

Es la documentación que **más se lee** y la que más se descuida. Estructura mínima:

````
# GestorTareas

Gestor de tareas por línea de comandos. Proyecto del módulo Entornos de Desarrollo (1.º DAW).

## Requisitos
- Node.js 20 o superior

## Instalación
```bash
git clone https://github.com/usuario/gestor-tareas.git
cd gestor-tareas
npm install
```

## Uso
| Comando | Descripción |
|---|---|
| `npm start -- añadir "Estudiar UD4"` | Añade una tarea |
| `npm start -- listar` | Lista todas |
| `npm start -- hecha 3` | Marca la tarea 3 como hecha |

## Desarrollo
- `npm run lint` — análisis estático
- `npm test` — pruebas
- `npm run docs` — genera la documentación

## Licencia
MIT
````

Práctica

P4.3 — Documenta el proyecto (2 h)

1. Documenta con JSDoc **todas** las funciones y clases de `gestor-tareas`: parámetros, tipos, retorno, excepciones y al menos tres `@example`.
2. Define un `@typedef` para el objeto tarea.
3. Configura `jsdoc.json` y genera la documentación en `docs/`.
4. Escribe un `README.md` completo con la estructura de arriba.
5. Añade `docs/` al `.gitignore`… o no: **decide y justifica** si la documentación generada debe versionarse o generarse en cada despliegue.
6. **Comprueba el efecto colateral**: pon el cursor sobre una de tus funciones al llamarla. ¿Qué muestra ahora VS Code que antes no mostraba?

7 Control de versiones

7.1 El problema que resuelve

Sin control de versiones:

```
proyecto/
├── index_v1.js
├── index_v2.js
├── index_v2_bueno.js
├── index_FINAL.js
├── index_FINAL_definitivo.js
└── index_FINAL_definitivo_ahora_si.js
```

Con control de versiones: **un solo fichero** y todo su historial guardado, con quién cambió qué, cuándo y por qué.

7.2 Conceptos

Término	Qué es
Repositorio	El proyecto + todo su historial
Commit	Una foto del proyecto en un momento, con mensaje y autor
Rama (*branch*)	Una línea de desarrollo paralela
HEAD	Dónde estás ahora
Fusión (*merge*)	Unir dos ramas
Conflicto	Dos ramas cambiaron la misma línea: hay que decidir
Remoto	Copia del repositorio en un servidor (GitHub, GitLab)
Clonar	Traerse un repositorio completo
Etiqueta (*tag*)	Marca sobre un commit; se usa para versiones (`v1.0.0`)

7.3 Centralizado vs distribuido

	Centralizado (CVS, Subversion)	Distribuido (Git, Mercurial)
Dónde está el historial	Solo en el servidor	**En cada copia**
Trabajar sin conexión	No	Sí, todo salvo sincronizar
Si cae el servidor	Se para todo	Cada uno tiene el historial completo
Ramas	Caras y lentas	Baratas e instantáneas
Cuota de mercado hoy	Residual	Git, abrumadoramente

7.4 Git: los comandos que necesitas

7.4.1 Configuración inicial (una vez por ordenador)

```
git config --global user.name "Nombre Apellidos"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main
git config --global core.autocrlf input     # en Linux/Mac
git config --global core.autocrlf true      # en Windows
```

7.4.2 Ciclo básico

```
git init                    # crear repositorio
git status                  # ¿en qué estado estoy?
git add fichero.js          # preparar cambios (zona de preparación / staging)
git add .                   # preparar todo
git commit -m "Añade el comando listar"
git log --oneline --graph --all
git diff                    # cambios no preparados
git diff --staged           # cambios preparados
```

7.4.3 Los tres estados de Git

```
Directorio de trabajo  →  Zona de preparación  →  Repositorio
       (modified)              (staged)              (committed)
            ── git add ──→          ── git commit ──→
```

7.4.4 Deshacer

Situación	Comando
Descartar cambios de un fichero no preparado	`git restore fichero.js`
Quitar de la zona de preparación	`git restore --staged fichero.js`
Corregir el último commit (mensaje o contenido)	`git commit --amend`
Deshacer un commit **creando otro** (seguro, público)	`git revert <hash>`
Mover HEAD hacia atrás (peligroso si ya lo subiste)	`git reset --hard <hash>`
Guardar temporalmente lo que tienes a medias	`git stash` / `git stash pop`

Ojo

`git revert` y `git reset` **no** son lo mismo. `revert` crea un commit nuevo que deshace; el historial queda intacto y es seguro en ramas compartidas. `reset --hard` **borra** historial: si esa rama ya está en el remoto y otros la han descargado, vas a provocar un desastre. Regla: *nunca reescribas historia que ya es pública*.

7.4.5 Ramas

```
git branch                      # listar
git switch -c feature/etiquetas  # crear y cambiar (moderno)
git checkout -b feature/etiquetas  # lo mismo (clásico)
git switch main
git merge feature/etiquetas
git branch -d feature/etiquetas  # borrar una vez fusionada
```

7.4.6 Conflictos

Cuando dos ramas cambian la misma línea, Git se para y marca el fichero:

```
<<<<<<< HEAD
const SALUDO = 'Hola';
=======
```

```
const SALUDO = 'Buenas';
>>>>>>> feature/saludos
```

Para resolverlo: editas el fichero dejando lo que debe quedar (y **borrando las marcas**), `git add fichero.js` y `git commit`. VS Code ofrece botones *Aceptar actual / Aceptar entrante / Aceptar ambos / Comparar cambios*, que son mucho más cómodos.

7.4.7 Remoto

```
git remote add origin https://github.com/usuario/gestor-tareas.git
git push -u origin main       # primera vez
git push                      # siguientes
git pull                      # traer y fusionar
git fetch                     # traer sin fusionar
git clone <url>               # copiar un repositorio completo
```

7.5 Mensajes de commit

Un historial legible vale tanto como el código. La convención más extendida es **Conventional Commits**:

```
<tipo>(<ámbito opcional>): <descripción en imperativo>

feat(tareas): añade el comando para etiquetar
fix(almacen): corrige la pérdida de datos al guardar sin carpeta data
docs(readme): documenta los scripts de npm
refactor(formato): extrae formatearTarea de formatearLista
test(tarea): añade casos límite de estaVencida
chore(deps): actualiza eslint a 9.10
```

Reglas prácticas:

- En **imperativo**: «añade», no «añadido» ni «añadiendo».
- Menos de 50 caracteres en la primera línea.
- Explica **el porqué** en el cuerpo si hace falta.
- Un commit = **un cambio lógico**. No mezcles refactorización con nueva funcionalidad.

Idea clave

Un mal mensaje de commit: `cambios`, `arreglos`, `asdf`, `ya funciona`, `subida final`. Dentro de tres meses, cuando busques cuándo se rompió algo, esos mensajes no te dirán nada y tendrás que abrir los 40 commits uno a uno.

7.6 Flujos de trabajo

7.6.1 GitHub Flow (el que usaremos)

1. `main` siempre debe estar funcionando.
2. Para cada cosa nueva, se crea una rama desde `main`.
3. Se trabaja y se hacen commits en esa rama.
4. Se abre un **Pull Request** (PR).
5. Alguien lo revisa y comenta.
6. Se fusiona a `main` y se borra la rama.

7.6.2 Git Flow (clásico, más pesado)

Ramas `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`. Adecuado para productos con versiones publicadas; excesivo para casi todo lo demás.

7.7 Git dentro de VS Code

La pestaña *Control de código fuente* (`Ctrl+Shift+G`) permite hacer casi todo sin terminal: ver cambios línea a línea, preparar por bloques, commit, cambiar de rama, resolver conflictos. Con **GitLens** además ves, en cada línea, quién la escribió y en qué commit (`git blame` integrado).

Idea clave

Aprende Git **por terminal** aunque luego uses los botones. El día que algo se tuerza —y se tuerce— la interfaz gráfica te va a dejar tirado y necesitarás entender qué está pasando.

Práctica

P4.4 — Git de cero a remoto (1 h)
Sobre `gestor-tareas`, ejecuta y documenta con capturas:

1. `git init` y configura tu nombre y correo.
2. Crea un `.gitignore` correcto. Comprueba con `git status` que `node_modules` no aparece.
3. Primer commit con todo el proyecto.
4. Haz tres cambios pequeños y tres commits, con mensajes en formato Conventional Commits.
5. `git log --oneline --graph`. Pega la salida.
6. Crea la rama `feature/etiquetas`, implementa el soporte de etiquetas, haz 2 commits.
7. Vuelve a `main`, fusiona y borra la rama.
8. Crea un repositorio en GitHub y sube todo.
9. Etiqueta la versión: `git tag -a v0.1.0 -m "Primera versión"` y `git push --tags`.
10. Usa `git revert` para deshacer uno de tus commits, y explica en qué se diferencia de `git reset`.

Práctica

P4.5 — El conflicto (1 h, por parejas)

1. La persona A crea un repositorio en GitHub y añade a B como colaborador.
2. Ambas clonan.
3. **Sin avisarse**, A y B modifican la **misma función** de `src/index.js` de forma distinta, y además cada una modifica un fichero diferente.
4. A hace `git push`. B intenta `git push` → error.
5. B hace `git pull` → **conflicto**.
6. B lo resuelve (usando la interfaz de VS Code), hace commit y push.
7. A hace `git pull` y comprueba el resultado.

Entrega: capturas de cada paso, el contenido del fichero con las marcas de conflicto antes de resolver, y una explicación de por qué solo hubo conflicto en uno de los dos ficheros modificados.

8 Integración continua

8.1 Qué es

Cada vez que alguien sube código, un servidor automáticamente descarga el proyecto, instala dependencias, ejecuta el analizador estático y **pasa todas las pruebas**. Si algo falla, avisa inmediatamente.

Ventaja: los errores se detectan en minutos, no en la próxima entrega, y nadie puede fusionar código que rompe la rama principal.

8.2 GitHub Actions

`.github/workflows/ci.yml`:

```yaml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  pruebas:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4

      - name: Preparar Node ${{ matrix.node }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Analizar el código
        run: npm run lint

      - name: Ejecutar pruebas con cobertura
        run: npm test -- --coverage
```

La `matrix` ejecuta el mismo trabajo en Node 20 y 22: es una prueba de compatibilidad gratis.

Idea clave

`npm ci` en lugar de `npm install`: en integración continua quieres la instalación **exactamente reproducible** del `package-lock.json`, no una que pueda traerte una versión nueva y hacer que el resultado dependa del día. (Lo vimos en UD2.)

Práctica

P4.6 — Entrega evaluable de la unidad (evaluable)
Repositorio en GitHub que contenga:

1. `gestor-tareas` **refactorizado**: sin duplicados, funciones cortas, nombres claros, sin números mágicos.
2. ESLint y Prettier configurados, y `npx eslint src` **sin errores**.
3. Documentación JSDoc completa y web generada.
4. `README.md` completo.
5. Historial de al menos **15 commits** con mensajes convencionales y significativos.
6. Al menos **una rama** creada, con **un Pull Request** abierto, revisado por un compañero (al menos un comentario real) y fusionado.
7. Una etiqueta `v1.0.0`.
8. `.github/workflows/ci.yml` funcionando, con la insignia verde en el `README.md`, ejecutando ESLint y **la batería de pruebas que entregaste en UD3**.
9. Un Pull Request donde se vea que la CI ha **bloqueado** un cambio que rompía las pruebas (haz que falle a propósito y luego arréglalo).

Rúbrica

Criterio	Peso	Nivel alto	Nivel medio	Nivel bajo
Refactorización	20 %	Olores eliminados y justificados, pasos visibles en el historial	Código mejorado	Apenas cambios
Análisis estático y formato	15 %	Sin avisos, configuración razonada	Configurado, quedan avisos	Sin configurar
Documentación JSDoc	15 %	Completa, con tipos, ejemplos y typedef	La mayoría documentada	Suelta o ausente
README	10 %	Completo y claro	Básico	Ausente
Uso de Git	20 %	Historial limpio, commits atómicos y bien escritos, ramas, tag	Commits correctos	«cambios», «asdf»
Pull Request y revisión	5 %	PR con descripción y revisión de fondo	PR sin revisión real	Sin PR
Integración continua	15 %	CI verde y demostración de que bloquea un fallo	CI verde	No hay

9 Resumen de la unidad

Concepto	En una línea
Deuda técnica	Atajo de hoy que se paga con intereses en cada cambio futuro
Code smell	Síntoma de un problema de diseño; el código funciona igualmente
Refactorizar	Cambiar la estructura interna sin cambiar el comportamiento externo
Regla de oro	Nunca refactorices sin pruebas automáticas
Regla de tres	A la tercera repetición, refactoriza
Extraer función	El patrón más usado: dar nombre a un trozo de código
Cláusula de guarda	Salir pronto para evitar anidamiento
F2 vs `Ctrl+H`	Renombrado semántico vs buscar y reemplazar ciego
Análisis estático	Detectar problemas sin ejecutar el código (ESLint)

Concepto	En una línea
Complejidad ciclomática	1 + puntos de decisión; también, mínimo de casos de prueba
Prettier	Formateador: decide el aspecto, no la corrección
JSDoc / Javadoc	Documentación en el código, generable a web
`@param`, `@returns`, `@throws`, `@example`	Las cuatro etiquetas imprescindibles
Comentario útil	Explica el **porqué**, no el **qué**
Repositorio	Proyecto + historial completo
Commit	Foto con autor, fecha y mensaje
Centralizado vs distribuido	Historial en el servidor vs en cada copia
Zona de preparación	El paso intermedio entre modificar y confirmar (`git add`)
`revert` vs `reset`	Deshacer creando historia vs borrar historia
Conflicto	Dos ramas cambian la misma línea; lo resuelve una persona
Conventional Commits	`tipo(ámbito): descripción en imperativo`
Integración continua	El servidor ejecuta lint y pruebas de UD3 en cada push
GitHub Flow	`main` estable + rama por función + Pull Request

10 Autoevaluación

1. ¿Es refactorizar lo mismo que optimizar? ¿Y que corregir un error?
2. Da tres razones por las que refactorizar sin pruebas es peligroso.
3. Calcula la complejidad ciclomática de una función con dos `if`, un `for` y un &&.
4. ¿Qué diferencia hay entre ESLint y Prettier? ¿Se solapan?
5. Escribe un comentario inútil y otro útil sobre la misma línea de código.
6. ¿Qué hace `@param {string} [nombre='anónimo']` exactamente?
7. Has hecho commit y push de algo que rompe `main`. ¿`revert` o `reset --hard`? ¿Por qué?
8. Explica los tres estados de un fichero en Git y los comandos que lo mueven entre ellos.
9. Tu compañero y tú habéis editado la misma línea. ¿Quién resuelve el conflicto y cuándo?
10. ¿Por qué `package-lock.json` sí va a Git y `node_modules/` no?